

Computational Thinking with Python

Session 4 – Data Structures

STUDENTS WILL LEARN TO ORGANIZE AND PROCESS DATA EFFICIENTLY

At the end of this session, you should be able to comprehend, trace, adapt and create Python code that:

- Creates, accesses, and modifies **lists**
- Uses list **methods** like `append()`, `sort()`, `pop()`
- Creates and uses **dictionaries** with key-value pairs
- Applies **string methods** for data cleaning (`split()`, `strip()`, `join()`)
- **Reads** and **writes** files in Python

AGENDA

- 1 Lists
- 2 Dictionaries
- 3 String Methods
- 4 Files

DATA STRUCTURES ENABLE EFFICIENT ORGANIZATION AND ACCESS TO DATA

Programming consists of:

- **Algorithms** – Steps to solve a problem
- **Data Structures** – Ways of organizing data

Why do we need data structures?

- Store multiple values in one variable
- Organize data logically
- Access and modify data efficiently

Python's main collections:

List

- Ordered collection of values
- Access by position (index)

Dictionary

- Collection of key-value pairs
- Access by key (label)

Collections store multiple values!

AGENDA

- 1 **Lists**
- 2 Dictionaries
- 3 String Methods
- 4 Files

LISTS STORE MULTIPLE ITEMS IN A SINGLE ORDERED VARIABLE

- Lists are used to store multiple items in a single variable
- Lists are surrounded by square brackets []
- Elements are separated by commas
- A list can contain any Python object – even other lists
- A list can be **empty**

```
list1 = [1, 2, 5, 9]           # integers
list2 = ["red", "yellow", "blue"] # strings
list3 = [1, "yellow", [1, 3, 5]] # mixed types
list4 = []                     # empty list
```

FOR LOOPS ITERATE THROUGH EACH ELEMENT OF A LIST

Code:

We already used lists with for loops:

```
friends = ["Joe", "Glenn",  
           "Sally"]  
  
for friend in friends:  
    print(f"Hello {friend}")  
  
print("Done!")
```

Output:

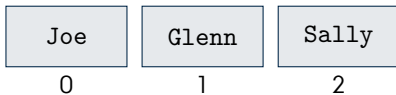
```
Hello Joe  
Hello Glenn  
Hello Sally  
Done!
```

For loops visit each element!

LIST INDEXING STARTS AT ZERO, NOT ONE

Key concepts:

- Access elements using an index in square brackets
- Index starts at **0** (not 1!)
- Negative indices count from the end



Examples:

```
friends = ["Joe", "Glenn",  
           "Sally"]  
print(friends[0])  
> Joe  
print(friends[2])  
> Sally  
print(friends[-1]) # Last element  
> Sally
```

LISTS CAN BE CHANGED AFTER CREATION, UNLIKE STRINGS

Mutable means we can change elements:

```
lotto = [2, 14, 26, 31, 40, 42]
print(lotto)
> [2, 14, 26, 31, 40, 42]

lotto[2] = 28 # Change element
print(lotto)
> [2, 14, 28, 31, 40, 42]
```

Compare with strings (immutable):

```
fruit = "Banana"
fruit[0] = "b"
> TypeError: strings are
immutable!
```

Lists can be changed; strings cannot!

LEN() AND RANGE() ARE ESSENTIAL FOR WORKING WITH LISTS

`len()` returns the number of elements:

```
items = [1, 2, "joe", 99]
print(len(items))
> 4
```

`range()` generates a sequence of numbers:

```
print(list(range(4)))
> [0, 1, 2, 3]
```

Two ways to iterate:

```
# Method 1: Direct iteration
for friend in friends:
    print(friend)
```

```
# Method 2: Index-based
for i in range(len(friends)):
    print(friends[i])
```

Method 1 is usually preferred!

SLICING EXTRACTS A PORTION OF A LIST INTO A NEW LIST

Extract a portion of a list using slicing:

```
list[start:end]
```

- start – Index to begin (included)
- end – Index to stop (not included)
- Omit start to start from beginning
- Omit end to go to the end

```
nums = [9, 41, 12, 3, 74, 15]
print(nums[1:3])
> [41, 12]

print(nums[:3]) # First 3
> [9, 41, 12]

print(nums[3:]) # From index 3
> [3, 74, 15]

print(nums[:]) # Copy all
> [9, 41, 12, 3, 74, 15]
```

Slicing creates a new list!

LIST METHODS MODIFY THE LIST DIRECTLY

```
stuff = []  
stuff.append("book")  
stuff.append(99)  
print(stuff)  
> ['book', 99]
```

append() adds items to the end!

Key methods:

- `append(x)` – Add `x` to end
- `insert(i, x)` – Insert at index
- `remove(x)` – Remove first `x`
- `pop()` – Remove last item
- `sort()` – Sort in place
- `reverse()` – Reverse in place

THE 'IN' OPERATOR CHECKS MEMBERSHIP, SORT() ORDERS ELEMENTS

Check if item is in list:

```
some = [1, 9, 21, 10, 16]
print(9 in some)
> True

print(15 in some)
> False

print(20 not in some)
> True
```

Sorting a list:

```
friends = ["Joe", "Glenn",
           "Sally"]
friends.sort()
print(friends)
> ['Glenn', 'Joe', 'Sally']
```

sort() changes the original list!

BUILT-IN FUNCTIONS SIMPLIFY COMMON LIST OPERATIONS

```
nums = [3, 41, 12, 9, 74, 15]
print(len(nums))    # Count
> 6

print(max(nums))    # Maximum
> 74

print(min(nums))    # Minimum
> 3

print(sum(nums))    # Total
> 154
```

Calculate average:

```
avg = sum(nums) / len(nums)
print(avg)
> 25.666...
```

Remember from loops?

These built-in functions are much simpler than writing your own loops!

Use built-in functions when possible!

APPLY LIST CONCEPTS THROUGH HANDS-ON PRACTICE

Practice: Open 01_Lists in your Codespace

Complete: 01_predict.py - 09_make.py

Predict → Investigate → Modify → Make

AGENDA

- 1 Lists
- 2 Dictionaries**
- 3 String Methods
- 4 Files

DICTIONARIES STORE DATA AS LABELED KEY-VALUE PAIRS

- Python's most powerful data collection
- Store data as **key-value pairs**
- Use curly braces {} and colons :
- Keys must be unique

Access values by key, not position!

```
stock = {}  
stock["apple"] = 50  
stock["bread"] = 20  
  
print(stock)  
> {'apple': 50, 'bread': 20}  
  
print(stock["apple"])  
> 50
```

LISTS USE POSITIONAL INDICES, DICTIONARIES USE NAMED KEYS

List – Access by index (position):

```
lst = []  
lst.append(21)  
lst.append(183)  
print(lst)  
> [21, 183]  
  
lst[0] = 23  
print(lst)  
> [23, 183]
```

Dictionary – Access by key (label):

```
student = {}  
student["age"] = 21  
student["course"] = 183  
print(student)  
> {'age': 21, 'course': 183}  
  
student["age"] = 23  
print(student)  
> {'age': 23, 'course': 183}
```

THE GET() METHOD SAFELY ACCESSES DICTIONARY VALUES

Problem: Accessing a missing key causes an error:

```
stock = {"apple": 10}
print(stock["banana"])
> KeyError: 'banana'
```

Solution: Use `get()` with a default value:

```
print(stock.get("banana", 0))
> 0
```

Counting pattern:

```
counts = {}
sales = ["apple", "bread",
         "apple", "milk"]

for item in sales:
    counts[item] =
counts.get(item, 0) + 1

print(counts)
> {'apple': 2, 'bread': 1,
   'milk': 1}
```

DICTIONARY ITERATION PROVIDES ACCESS TO KEYS, VALUES, OR BOTH

Loop through keys:

```
prices = {"apple": 1.50, "bread": 2.00}

for item in prices:
    print(f"{item}: {prices[item]}")

> apple: 1.5
> bread: 2.0
```

Useful dictionary methods:

```
prices.keys()    # All keys
prices.values()  # All values
prices.items()   # Key-value pairs
```

```
for k, v in prices.items():
    print(f"{k} costs {v}")
```

By default, loops iterate over keys!

APPLY DICTIONARY CONCEPTS THROUGH HANDS-ON PRACTICE

Practice: Open `02_Dictionaries` in your Codespace

Complete: `01_predict.py` - `07_make.py`

Predict → Investigate → Modify → Make

AGENDA

- 1 Lists
- 2 Dictionaries
- 3 String Methods**
- 4 Files

REAL-WORLD DATA REQUIRES CLEANING BEFORE ANALYSIS

Real-world data is messy!

- Extra spaces, inconsistent capitalization, mixed formats
- Data often comes as text that needs to be parsed and cleaned
- String methods help transform raw data into usable information

Example – Raw data from a file:

```
" Müller, Hans ; München ; 85000 "
```

What we need:

```
name = "Müller, Hans"  city = "München"  salary = 85000
```

STRIP() REMOVES LEADING AND TRAILING WHITESPACE

strip() removes leading and trailing whitespace:

```
text = " Hello, World! "  
print(text.strip())  
> Hello, World!  
  
print(text.lstrip()) # Left only  
> Hello, World!  
  
print(text.rstrip()) # Right only  
> Hello, World!
```

Common use case:

```
# Clean user input  
user_input = input("Name: ")  
name = user_input.strip()  
  
# Clean data from file  
line = " data here \n"  
clean = line.strip()
```

Always strip() input and file data!

SPLIT() BREAKS A STRING INTO A LIST OF PARTS

`split()` breaks a string into a list:

```
# Split on whitespace (default)
text = "Hello World Python"
words = text.split()
print(words)
> ['Hello', 'World', 'Python']

# Split on specific character
data = "apple,banana,cherry"
fruits = data.split(",")
print(fruits)
> ['apple', 'banana', 'cherry']
```

Parsing data:

```
# Parse a CSV-like line
line = "Müller;München;85000"
parts = line.split(";")

name = parts[0]
city = parts[1]
salary = int(parts[2])

print(f"{name} from {city}")
> Müller from München
```

JOIN() COMBINES LIST ELEMENTS INTO A SINGLE STRING

`join()` combines a list into a string:

```
# Join with separator
words = ["Hello", "World"]
sentence = " ".join(words)
print(sentence)
> Hello World

# Join with different separators
print("-".join(["a", "b", "c"]))
> a-b-c

print(", ".join(["apple",
"banana"]))
> apple, banana
```

Note the syntax:

`separator.join(list)`

- The separator is a string
- It goes between each element
- The list must contain strings

`split()` and `join()` are opposites!

CASE METHODS ENABLE CONSISTENT STRING COMPARISONS

```
text = "Hello World"
print(text.lower())
> hello world

print(text.upper())
> HELLO WORLD

print(text.title())
> Hello World

print(text.capitalize())
> Hello world
```

Use case – Case-insensitive comparison:

```
answer = input("Continue? (y/n): ")
if answer.lower() in ("y", "yes"):
    print("Continuing...")
```

This accepts "Y", "y", "YES", "yes", etc.

Use lower() for consistent checks!

REPLACE() SUBSTITUTES TEXT THROUGHOUT A STRING

`replace()` substitutes text:

```
text = "Hello World"
new_text = text.replace("World",
                        "Python")
print(new_text)
> Hello Python

# Remove characters
phone = "123-456-7890"
clean = phone.replace("-", "")
print(clean)
> 1234567890
```

Key points:

- Returns a new string
- Original string is unchanged
- Replaces all occurrences

```
text = "aaa"
print(text.replace("a", "b"))
> bbb
```

STRING METHODS RETURN NEW STRINGS AND LEAVE THE ORIGINAL UNCHANGED

Method	Purpose	Example
<code>strip()</code>	Remove whitespace	<code>" hi ".strip() → "hi"</code>
<code>split(sep)</code>	String → List	<code>"a,b".split(",") → ["a","b"]</code>
<code>sep.join(list)</code>	List → String	<code>"-".join(["a","b"]) → "a-b"</code>
<code>lower()</code>	Lowercase	<code>"Hi".lower() → "hi"</code>
<code>upper()</code>	Uppercase	<code>"Hi".upper() → "HI"</code>
<code>replace(old,new)</code>	Replace text	<code>"Hi".replace("H","B") → "Bi"</code>

String methods return new strings – they don't modify the original!

APPLY STRING METHODS THROUGH HANDS-ON PRACTICE

Practice: Open 03_StringMethods in your Codespace

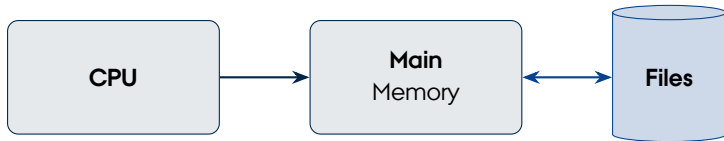
Complete: 01_predict.py - 09_make.py

Predict → Investigate → Modify → Make

AGENDA

- 1 Lists
- 2 Dictionaries
- 3 String Methods
- 4 Files**

FILES PROVIDE PERMANENT STORAGE THAT PERSISTS AFTER PROGRAM EXECUTION



- Files store data permanently on disk (secondary memory)
- Programs read files into memory, process them, and write results back
- Python uses file handlers to interact with files

THE 'WITH' STATEMENT ENSURES FILES ARE PROPERLY CLOSED

Basic file reading:

```
file = open("mydata.txt")
for line in file:
    print(line.strip())
file.close()
```

Better approach with 'with':

```
with open("mydata.txt") as file:
    for line in file:
        print(line.strip())
```

Why use 'with'?

- Automatically closes the file
- Even if an error occurs
- Cleaner, safer code

Read all at once:

```
content = f.read()    # One string
lines = f.readlines() # List
```

FILE MODE 'W' OVERWRITES WHILE 'A' APPENDS TO EXISTING CONTENT

Write to a file:

```
data = ["Hello", "World"]
with open("out.txt", "w") as f:
    for line in data:
        f.write(line + "\n")
```

File modes:

- "r" - Read (default)
- "w" - Write (overwrites!)
- "a" - Append

Important notes:

- "w" overwrites existing content!
- Add "\n" for new lines
- Use "a" to add to existing file

WARNING: This overwrites!
`open("file.txt", "w")`

THE CSV MODULE PARSES COMMA-SEPARATED VALUES INTO LISTS

CSV = Comma-Separated Values

```
import csv

with open("data.csv") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

Each row is a list of values.

Example CSV file:

```
name,city,salary
Homer,Springfield,50000
Marge,Springfield,45000
```

Output:

```
['name', 'city', 'salary']
['Homer', 'Springfield', '50000']
['Marge', 'Springfield', '45000']
```

APPLY FILE OPERATIONS THROUGH HANDS-ON PRACTICE

Practice: Open 04_Files in your Codespace

Complete: 01_predict.py - 10_make.py

Predict → Investigate → Modify → Make

CHOOSE LISTS FOR ORDERED SEQUENCES, DICTIONARIES FOR LABELED DATA

Use a List when:

- Order matters
- Access items by position (index)
- You have a sequence of similar items

Use a Dictionary when:

- Look up values by a label (key)
- Counting or grouping items
- You have key-value relationships

Key Takeaway: **List** = ordered, access by index; **Dictionary** = labeled, access by key

DATA STRUCTURES PROVIDE POWERFUL TOOLS FOR ORGANIZING INFORMATION

- ✓ **Lists** store ordered collections using `[]` and index access
- ✓ **List methods:** `append()`, `sort()`, `pop()`, `in` operator
- ✓ **Dictionaries** store key-value pairs using `{}` and key access
- ✓ **Dict methods:** `get()`, `keys()`, `values()`, `items()`
- ✓ **String methods:** `strip()`, `split()`, `join()`
- ✓ **Files:** Use `with open()` for safe reading and writing

Data structures organize data efficiently!

Thank You!

Nikolai Stein

